

ESCUELA:

Tecnológico Nacional de México campus Culiacán

Carrera:

Ingeniería en Sistemas Computacionales

Materia:

Tópicos de Inteligencia Artificial

Profesor(a):

Zuriel Dathan Mora Felix

Nombre de la tarea:

Tarea Bonus N Reynas

ALUMNOS:

22170574 Bejarano Ureta Sebastián

22170861 Garcia Inzunza Kevin Alejandro

FECHA:

09/10/2026

Culiacán Sinaloa

Descripción del problema

N Reynas

El problema de las “N reinas” es un desafío clásico algorítmico en el que el objetivo principal es calcular de cuántas maneras diferentes se pueden colocar n reinas en un tablero de dimensiones n x n. La restricción fundamental es que ninguna de ellas deba amenazarse entre sí, es decir, ninguna reina puede ocupar la misma fila, columna o diagonal, evitando así que una pueda capturar a otra en un solo movimiento.

En esta formulación, la variable n representa la misma cantidad en ambos parámetros, lo que significa que el número de reinas debe ser exactamente igual al número de filas y columnas del tablero (cuyo caso particular más conocido es el de colocar 8 reinas en un tablero estándar de 8 x 8) [1].

Búsqueda Tabú

La Búsqueda Tabú (Tabu Search) es una técnica metaheurística empleada para dirigir y regular un procedimiento de búsqueda local, con el objetivo de encontrar soluciones casi óptimas en problemas de optimización. Su concepto fundamental radica en restringir o prohibir movimientos hacia estados visitados previamente en áreas que no ofrecen una mejor solución, de ahí el uso del término “tabú”. En este contexto, las mejoras del sistema se producen al modificar la solución mediante movimientos estratégicos.

La idea clave de este método es evitar la revisión de soluciones ya exploradas y catalogadas como deficientes mediante el mantenimiento de una lista de soluciones “tabú”. En los algoritmos tradicionales, los nodos visitados se conservan en una lista CERRADA. Cuando se exploran nuevos nodos en el espacio de estados, aquellos previamente visitados se eliminan de la lista de candidatos (lista ABIERTA) tomando como referencia la lista CERRADA. Sin embargo, en la Búsqueda Tabú, la lista CERRADA actúa directamente como una lista TABÚ. Esto permite que los movimientos o estados restringidos puedan ser evaluados nuevamente si algún estado previo guardado en la lista resulta ser superior a los mejores movimientos [2].

Representación de P y S

Representación de la Posición o Estado (P): Se optó por no emplear una matriz tradicional de 8x8 para modelar el tablero, ya que esto haría el programa más lento y pesado de evaluar. En su lugar, el estado (P) se representa utilizando un arreglo simple de 8 espacios (una lista unidimensional).

La lógica aplicada es la siguiente:

- El **índice de la lista** (las posiciones del 0 al 7) indica la columna.

- El **número guardado** en ese índice indica la fila donde se encuentra la reina.

Por ejemplo, un estado inicial se ve así: [0, 4, 7, 5, 2, 6, 1, 3]. Al manejar siempre una secuencia (números del 0 al 7 sin repetirse), la misma estructura de la lista garantiza automáticamente que nunca habrá dos reinas en la misma fila ni en la misma columna. Esto representa una gran ventaja computacional, ya que el algoritmo no pierde tiempo revisando choques rectos.

Representación de la Solución (S): La solución (S) es el estado del tablero donde ya no existe ningún ataque entre las reinas; es decir, cuando la función que evalúa los conflictos llega a 0.

Como la estructura de la lista ya evita que las reinas choquen en horizontal y vertical, lo único que el algoritmo debe buscar son los ataques en diagonal. Para lograrlo, se evalúa la siguiente condición matemática:

$$|tablero[\,i\,] - tablero[\,j\,]| = |\,i - j|$$

En palabras sencillas: si la distancia que hay entre las columnas de dos reinas es exactamente igual a la distancia que hay entre sus filas, significa que están paradas en la misma diagonal y se están atacando.

Por lo tanto, se llega a la solución (S) cuando el algoritmo Tabú, después de realizar los intercambios y evitar quedarse ciclado gracias a su lista de memoria, encuentra un acomodo donde esta fórmula no detecta ningún choque.

Pseudo codigo:

1. Función para contar conflictos diagonales

```
Plaintext
Función contar_conflictos(tablero):
    conflictos = 0

    Para cada reina i desde 0 hasta 7:
        Para cada reina j desde i + 1 hasta 7:
            # Si la distancia vertical es igual a la horizontal, están en la misma diagonal
            Si valor_absoluto(tablero[i] - tablero[j]) == (j - i):
                conflictos = conflictos + 1

    Retornar conflictos
Fin Función
```

2. Función del Algoritmo de Búsqueda Tabú

Plaintext

```
Función algoritmo_tabu(tablero_inicial):
    tablero_actual = copia de tablero_inicial
    mejor_tablero = copia de tablero_actual
    menor_conflictos = contar_conflictos(tablero_actual)

    lista_tabu = lista vacía (memoria a corto plazo)
    iteraciones = 0
    iniciar_cronometro()

    # Repetir mientras queden conflictos y no pasemos de 500 intentos
    Mientras menor_conflictos > 0 Y iteraciones < 500:
        iteraciones = iteraciones + 1

        mejor_vecino = Nulo
        conflictos_vecino = 999
        movimiento_elegido = Nulo

        # Probar todos los intercambios de posición posibles
        Para i desde 0 hasta 7:
            Para j desde i + 1 hasta 7:
                vecino = intercambiar reinas en posiciones (i, j)
                conflictos_nuevos = contar_conflictos(vecino)
                movimiento = pareja (i, j)

                  es_prohibido = (movimiento está en lista_tabu) O (inverso de movimiento está en
lista_tabu)
                supera_el_record_global = (conflictos_nuevos < menor_conflictos)

                    # Regla: lo aceptamos si no está prohibido, o si rompe el récord (criterio de
aspiración)
                Si NO es_prohibido O supera_el_record_global:
                    Si conflictos_nuevos < conflictos_vecino:
                        conflictos_vecino = conflictos_nuevos
                        mejor_vecino = vecino
```

```
                    movimiento_elegido = movimiento

        # Si no encontramos ningún movimiento válido, detenemos el bucle
        Si mejor_vecino es Nulo:
            Romper bucle

        tablero_actual = mejor_vecino

        # Si este vecino supera el récord de la mejor solución que llevamos, lo guardamos
        Si conflictos_vecino < menor_conflictos:
            mejor_tablero = copia de tablero_actual
            menor_conflictos = conflictos_vecino

        # Guardar el movimiento en la lista tabú para no repetirlo pronto
        Agregar movimiento_elegido a lista_tabu
        Si tamaño de lista_tabu > 5:
            Eliminar el elemento más viejo de lista_tabu

    tiempo_total = detener_cronometro()
    Retornar mejor_tablero, menor_conflictos, iteraciones, tiempo_total
Fin Función
```

3. Función para imprimir el tablero y Programa Principal

```
Plaintext
Función mostrar_matriz(tablero, titulo):
    Escribir titulo
    Escribir números de columnas: "0 1 2 3 4 5 6 7"
    Para fila desde 0 hasta 7:
        Para col desde 0 hasta 7:
            Si tablero[col] == fila:
                Escribir "Q "  (reina)
            Sino:
                Escribir ". "  (casilla vacía)
        Saltar de línea
Fin Función
```

```
# --- PROGRAMA PRINCIPAL ---
Inicio
    Preguntar al usuario si quiere ingresar un tablero

    Si usuario responde 's':
        Leer posiciones del tablero
    Sino:
        Generar tablero aleatorio sin filas repetidas (permutación del 0 al 7)

    mostrar_matriz(tablero_inicial, "Tablero Inicial")

    resultado, total_conflictos, total_iteraciones, tiempo = algoritmo_tabu(tablero_inicial)

    mostrar_matriz(resultado, "Mejor Tablero Encontrado")

    Mostrar resultado
    Mostrar total_conflictos
    Mostrar total_iteraciones
    Mostrar tiempo
Fin
```

PD:

pudimos haber utilizado la librería de matplotlib para una mejor representación del tablero (para que se viera el tablero de ajedrez) pero no supimos usarla y por cuestiones de tiempo no lo utilizamos.

REFERENCIAS

[1] Brown, P. (2017, September 5). Solving the n-queens problem. *Medium*. https://medium.com/@paabrown/solving-the-n-queens-problem-6891e3d21b0

[2] Andhale, A. (2024, February 4). Tabu Search | Artificial intelligence. *Medium*. https://medium.com/@andy08/tabu-search-artificial-intelligence-9853f44e6923